# 25 Hard-Level SQL Server Query Exercises

**B.Tech Semester 1 | Questions, Database Schema, Sample Data and Solutions**

Topics covered: Aggregate Functions with GROUP BY and HAVING; JOIN operations; Window Functions including ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), SUM() OVER(), AVG() OVER() and NTILE().

## Part 1: Database Schema

The exercises use a College Management Database.

### 1. Create Database and Tables

```
CREATE DATABASE CollegeDB;
GO

USE CollegeDB;
GO

CREATE TABLE Department
(
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50)
);

CREATE TABLE Student
(
   StudentID INT PRIMARY KEY,
   StudentName VARCHAR(50),
   Gender VARCHAR(10),
   City VARCHAR(50),
   DeptID INT,
   AdmissionYear INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

CREATE TABLE Course
(
   CourseID INT PRIMARY KEY,
   CourseName VARCHAR(50),
   Credits INT,
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

CREATE TABLE Enrollment
(
   EnrollID INT PRIMARY KEY,
   StudentID INT,
   CourseID INT,
   Semester INT,
   Marks INT,
   FeePaid DECIMAL(10,2),
   FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
   FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
```

## 2. Insert Sample Data

```
INSERT INTO Department VALUES
(1,'Computer'),
(2,'IT'),
(3,'Mechanical'),
(4,'Civil'),
(5,'Electrical');


INSERT INTO Student VALUES
(101,'Amit','Male','Rajkot',1,2024),
(102,'Neha','Female','Ahmedabad',1,2024),
(103,'Rahul','Male','Rajkot',2,2023),
(104,'Priya','Female','Surat',2,2024),
(105,'Karan','Male','Rajkot',3,2023),
(106,'Pooja','Female','Ahmedabad',3,2023),
(107,'Jay','Male','Surat',1,2023),
(108,'Riya','Female','Rajkot',2,2024),
(109,'Meet','Male','Vadodara',4,2023),
(110,'Diya','Female','Surat',4,2024),
(111,'Harsh','Male','Ahmedabad',5,2023),
(112,'Nisha','Female','Rajkot',5,2024);


INSERT INTO Course VALUES
(201,'DBMS',4,1),
(202,'Programming',3,1),
(203,'Networking',3,2),
(204,'Web Development',4,2),
(205,'Thermodynamics',3,3),
(206,'Structural Design',4,4),
(207,'Power Systems',4,5),
(208,'Data Science',4,1);


INSERT INTO Enrollment VALUES
(1,101,201,1,85,4000),
(2,101,202,1,75,3000),
(3,101,208,2,92,4000),
(4,102,201,1,95,4000),
(5,102,202,1,88,3000),
(6,102,208,2,95,4000),
(7,103,203,1,78,3000),
(8,103,204,1,82,4000),
(9,104,203,1,90,3000),
(10,104,204,1,90,4000),
(11,105,205,1,65,3000),
(12,105,205,2,75,3000),
(13,106,205,1,85,3000),
(14,107,201,1,70,4000),
(15,107,202,1,80,3000),
(16,108,203,1,88,3000),
(17,108,204,1,76,4000),
(18,109,206,1,92,4000),
(19,110,206,1,82,4000),
(20,111,207,1,68,4000),
(21,112,207,1,94,4000),
(22,103,203,2,85,3000),
(23,104,204,2,95,4000),
(24,109,206,2,88,4000),
(25,111,207,2,78,4000);
```

# Part 2: 25 Hard-Level SQL Questions

## Section A – Aggregate Functions with GROUP BY (Q1–Q8)

**Q1.** Display the number of students in each department. Show only those departments that have more than 2 students.

**Q2.** Display the average marks of students in each department across all their courses. Show only departments whose average marks are greater than 80.

**Q3.** Display the total number of enrollments, maximum marks, minimum marks, and average marks for each course. Show only courses having at least 3 enrollments.

**Q4.** Find the number of distinct departments represented in each city. Display only cities having students from at least 2 different departments.

**Q5.** Calculate the total fee collection for each department. Display only departments whose total collection is greater than ₹15,000.

**Q6.** Find the average marks for each department in each semester. Display only department-semester combinations having an average greater than 80.

**Q7.** Display each student's total number of distinct courses and total marks. Show only students who have enrolled in at least 2 different courses.

**Q8.** Find the number of students scoring 80 or more marks in each department. Display only departments having at least 2 such students.

## Section B – JOIN Operations (Q9–Q16)

**Q9.** Display student name, department name, and course name. Show only students who have enrolled in the DBMS course.

**Q10.** Find students who have not enrolled in any course offered by their own department.

**Q11.** Display all department names along with their courses. Departments without any courses must also be displayed.

**Q12.** Find pairs of students who belong to the same department and the same city. Each pair should appear only once.

**Q13.** Find students who scored higher marks in Semester 1 than in Semester 2 in the same course.

**Q14.** Display each student's name and the number of courses they have enrolled in. Students who have not enrolled in any course must also be displayed.

**Q15.** Find students who scored more than 90 marks in a course offered by their own department.

**Q16.** Display the name of the department, the student who scored the highest marks in that department, and their marks. If multiple students have the same highest marks, display all of them.

## Section C – Window Functions (Q17–Q25)

**Q17.** Rank students within each department based on their average marks. The student with

the highest average should receive Rank 1.

**Q18.** Assign ROW_NUMBER() to students in each course based on their marks. The student with the highest marks should receive number 1.

**Q19.** Find the top 2 students in each department based on their average marks.

**Q20.** Display each student's semester-wise marks along with their previous semester's marks. Use the LAG() function.

**Q21.** Display each student's average marks along with the overall average marks of their department. Use AVG() OVER().

**Q22.** Display each student's marks along with their cumulative marks in that course. Use SUM() OVER().

**Q23.** Find the student who secured the second-highest average marks in each department. Use DENSE_RANK().

**Q24.** Find the students who secured the highest and lowest marks in each course using window functions.

**Q25.** Divide students in each department into four groups based on their average marks: Top 25%, Next 25%, Next 25%, and Bottom 25%. Use NTILE(4).